ShadowMem

Volatile Memory Forensics & Fileless Malware Detection - Prototype Summary

1. What It Is

ShadowMem is a **Windows volatile memory forensics and fileless malware detection** prototype built with Python (FastAPI + scikit-learn). It scans all running processes' virtual address space to detect injection-based attacks without relying on disk artifacts.

2. Architecture

Layer	File(s)	Role
Scanner	memoryscanner.py	Uses Win32 APIs (VirtualQueryEx, NtQueryInformationThread) via ctypes to walk every process's VAD tree, counting RWX pages, unbacked executable regions, threads starting in unbacked memory, and VAD protection flips (RW->RX/RWX).
ML Model	train_model.py -> shadowmem_model.pkl	Trains a 9-class HistGradientBoostingClassifier (RobustScaler + HGB pipeline) on synthetic forensic data. 7-feature vector: [rwx_count, unbacked_exec_count, total_rwx_bytes, num_threads, thread_unbacked_start, is_known_jit_parent, vad_protection_anomaly]. Includes canary sanity gates before saving.
Detection Engine	app.py (evaluate_process)	Feeds scanner output into the model, applies JIT allowlist suppression (browsers/Electron capped at 25%), managed runtime path validation (PowerShell only if under C:\Windows), and classifies into Clean / Suspicious (>=35%) / Malicious (>=70%).
SOC Dashboard	app.py (embedded HTML) + index.html	Cyberpunk-styled web UI with live scan results, threat posture gauge, VAD radar scope, process table with isolate/resume controls, and export-to-JSON.
WebSocket Demo	server.py + index.html	Synthetic telemetry/alert streaming for UI showcase (not real memory reads).

3. ML Feature Vector (7 Features)

#	Feature	Description
0	rwx_count	Count of PAGE_EXECUTE_READWRITE regions
1	unbacked_exec_count	MEM_PRIVATE executable regions with no backing file
2	total_rwx_bytes	Aggregate byte size of RWX allocations
3	num_threads	Total active threads in the process
4	thread_unbacked_start	# threads whose start address resolves into unbacked/private memory
5	is_known_jit_parent	1 if process is a known JIT-capable runtime, else 0
6	vad_protection_anomaly	Count of VAD regions transitioning RW -> RX/RWX

4. Multi-Class Model (9 Classes)

The model classifies processes into 5 benign and 4 malicious subclasses:

Class	Category	Description
benign_daemon (0)	Benign	Ordinary background system daemons/services
benign_gui (1)	Benign	Multi-threaded GUI applications (office, IDEs)
benign_jit (2)	Benign	JIT-compiling runtimes (Chromium/V8, Electron, JVM)
benign_managed (3)	Benign	Managed .NET runtimes (PowerShell, pwsh, dotnet)
benign_desktop (4)	Benign	Desktop apps outside JIT allowlist
mal_reflective (5)	Malicious	Reflective DLL Injection - T1055.001

Class	Category	Description
mal_hollow (6)	Malicious	Process Hollowing - T1055.012
mal_shellcode (7)	Malicious	Shellcode / C2 Beacon - T1620
mal_hijack (8)	Malicious	Thread / APC Execution Hijacking - T1055.003

5. Malicious Technique Forensic Signatures

Technique	MITRE ID	Key Forensic Signature
mal_reflective	T1055.001	3-5 unbacked regions, 1-2 unbacked thread starts, moderate VAD flips
mal_hollow	T1055.012	5-10 unbacked regions, large RWX bytes (full PE unpack), 5-8 VAD flips
mal_shellcode	T1620	1-3 small RWX regions, exactly 1 unbacked thread start, near-zero VAD flips
mal_hijack	T1055.003	Minimal RWX, zero unbacked threads (reuses existing thread), 2-4 VAD flips

6. Process Containment

- **Real suspend/resume** via POST /api/isolate/{pid} and POST /api/resume/{pid}
- Uses OpenThread + SuspendThread/ResumeThread (with NtSuspendProcess fallback)
- Blocklist protects critical system processes (csrss, lsass, services, etc.)
- In-memory audit trail logs every isolation action with timestamp

7. Test Harnesses

File	Description
test_injector.py	Safe in-process simulator: allocates RWX memory, performs RW->RWX VAD flip, spawns thread into unbacked memory (NOP sled + RET only, no actual shellcode)
malware_simulator.py	Extended simulator with 4 attack modes (shellcode, reflective DLL, process hollowing, thread hijack) that self-registers into live prototype via WebSocket/API
test_sandbox.py	Automated validation suite verifying detection accuracy across all attack modes and the 7-vector breakdown

8. Prototype Limitations

- Trained entirely on **synthetic data** - no real malware samples used
- **Windows-only** (Win32 ctypes calls, VAD walking requires Windows kernel structures)
- Suspend/resume requires sufficient process privileges (non-admin cannot suspend SYSTEM processes)
- The WebSocket demo (server.py) generates **synthetic telemetry** for UI testing, not real memory reads
- Snapshot semantics: a single scan can miss transient beacons

9. Technology Stack

Component	Technology
Language	Python 3.14 (Windows)
Web Framework	FastAPI + Uvicorn
ML Library	scikit-learn (HistGradientBoostingClassifier)
System APIs	Win32 via ctypes (VirtualQueryEx, NtQueryInformationThread, OpenThread)
Process Info	psutil
Frontend	Tailwind CSS CDN, JavaScript (WebSocket, Canvas API)
PDF Generation	reportlab

10. How to Run

Task	Command
Launch dashboard	launch_shadowmem.bat (or: .venv\Scripts\activate && python -m uvicorn app:app --reload)
Scan via API	curl http://127.0.0.1:8000/api/scan
CLI scan	python detecter.py
Retrain model	python train_model.py
Live-fire demo	Terminal 1: python test_injector.py. Terminal 2: python detecter.py
Safe malware sim	python malware_simulator.py